

Task 1: PPT Audit, Strategic Refinements, and Competitive Benchmarking

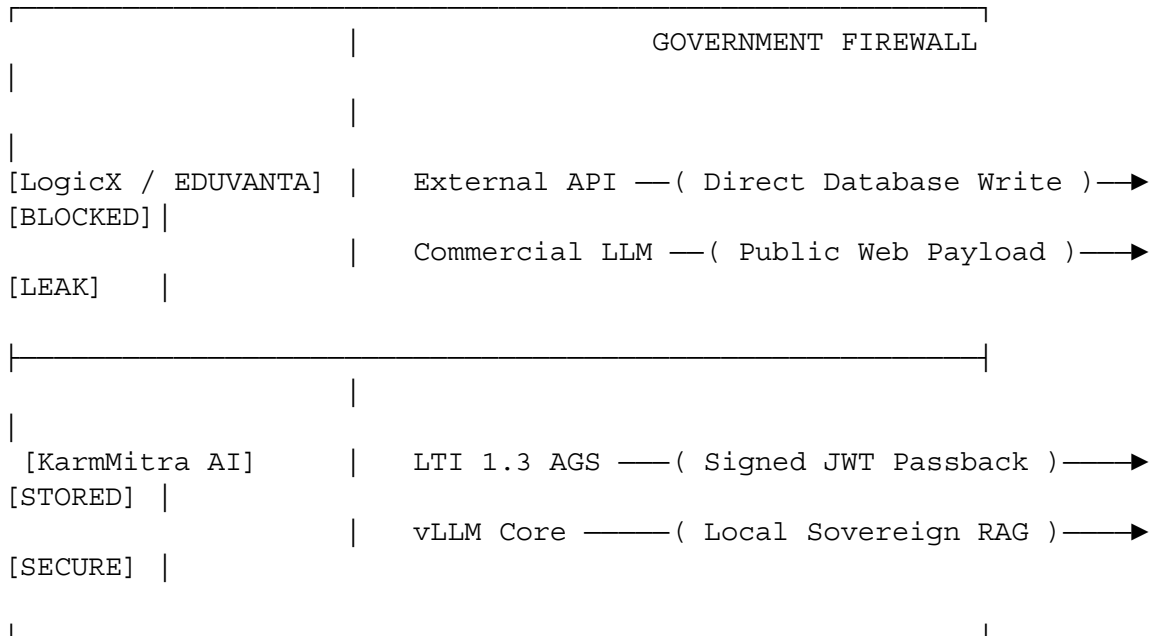
1. Critical Slide-by-Slide Audit & Add/Remove Matrix

Analyzing your internal pitch against your slide deck, the Hackathon Pitch Preparation Guide, the SIH Winner's Playbook, and rival decks (LogicX and EDUVANTA):

Slide & Title	What's Working	What Hurts Your Score	Required Addition / Removal / Change
Slide 1–2: Title & Mindset	Clear branding (KarmMitra AI), proper Problem Statement ID (SIH26101), clean theme.	Standard layout; does not immediately showcase national alignment.	Change: Add a 1-line sub-banner: "A Sovereign AI and LTI 1.3-Compliant Competency Engine for Mission Karmayogi & MoSPI."
Slide 3: Problem Analysis	Accurately identifies 4 distinct bottlenecks: generic catalog, cold-start gap, manual quiz creation, and data sovereignty risks.	Violates the 15-Second Rule & Hard Data mandates. Text-heavy bullet points with zero empirical data or domain division callouts.	Add: Real statistics: "4,600+ courses, 1.5 Cr+ registered civil servants, 100+ manual trainer hours wasted per NSSTA cycle, 0% data localization on commercial APIs." Name the exact MoSPI divisions affected: NSO, FOD, SDRD, and NAD.
Slide 4: Proposed Solution	4 clear pillars (iGOT Plugin, Skill Checker, Local AI Quiz Maker, Private Career Paths).	The visual layout reads like an essay rather than a modular product framework.	Change: Convert the 4 text blocks into 4 structured interface cards. Replace informal phrasing ("Officials don't need a new password") with professional architecture terms: "LTI 1.3 Single Sign-On (SSO) with zero database credentials required".
Slide 5: Architecture & Workflow	Explicitly outlines the 4-step user journey (Handshake → Triage → RAG Generation →	Purely descriptive text; lacks a formal data-flow block diagram (Frontend → API	Add: A concrete flowchart with directional arrows showing user tokens and data packets

Slide & Title	What's Working	What Hurts Your Score	Required Addition / Removal / Change
	70:20:10 Pathway Sync).	Gateway \leftarrow Vector Engine \rightarrow iGOT AGS).	flowing between React \rightarrow FastAPI \rightarrow vLLM/ChromaDB \rightarrow iGOT Assignment & Grade Services (AGS).
Slide 6: Technical Stack	Production-grade stack (React 18, FastAPI, Llama-3-8B via vLLM, ChromaDB, PostgreSQL, Bhashini).	Rivals (EDUVANTA, LogicX) also list FastAPI/React. Does not highlight why your choices are strictly required for security.	Add: Trade-off annotations: <i>"vLLM for 4x inference throughput on mid-tier GPUs; PostgreSQL for relational KCM schemas vs. ChromaDB for semantic vector search"</i> .
Slide 7: Feasibility & Viability	4 clear pillars (Technical, Financial, Market/Adoption, Operational).	Uses redundant numbering (01, 02, 03, 04 repeated twice) and lacks a clear timeline execution roadmap.	Add: Insert a 6-Week Implementation Roadmap with phased deployment milestones (Phase 1: NSSTA/ISS Pilot; Phase 2: SSS Rollout; Phase 3: Pan-India State Statistical Bureaus).
Slide 8: Impact & Benefits	Clear dual-stakeholder breakdown (Officials vs. Government/MoSPI).	Uses descriptive statements (" <i>Reduced training gaps</i> ") instead of quantified metrics.	Change: Quantify outcomes: <i>"80% reduction in course discovery friction, 90% reduction in manual assessment drafting time for NSSTA, 100% data sovereignty on MeghRaj/NIC servers."</i>
Slide 9: Comparison & References	Direct matrix against Generic iGOT and Commercial SaaS; solid citations (DoPT, MeitY, NeurIPS 2020).	EDUVANTA also includes Coursera and Degreed in their table.	Add: Add an "Architecture Integrity" row: Explicitly show that KarmMitra AI uses Zero Direct Database Writes via LTI 1.3 AGS, whereas others rely on unauthorized REST API writes.

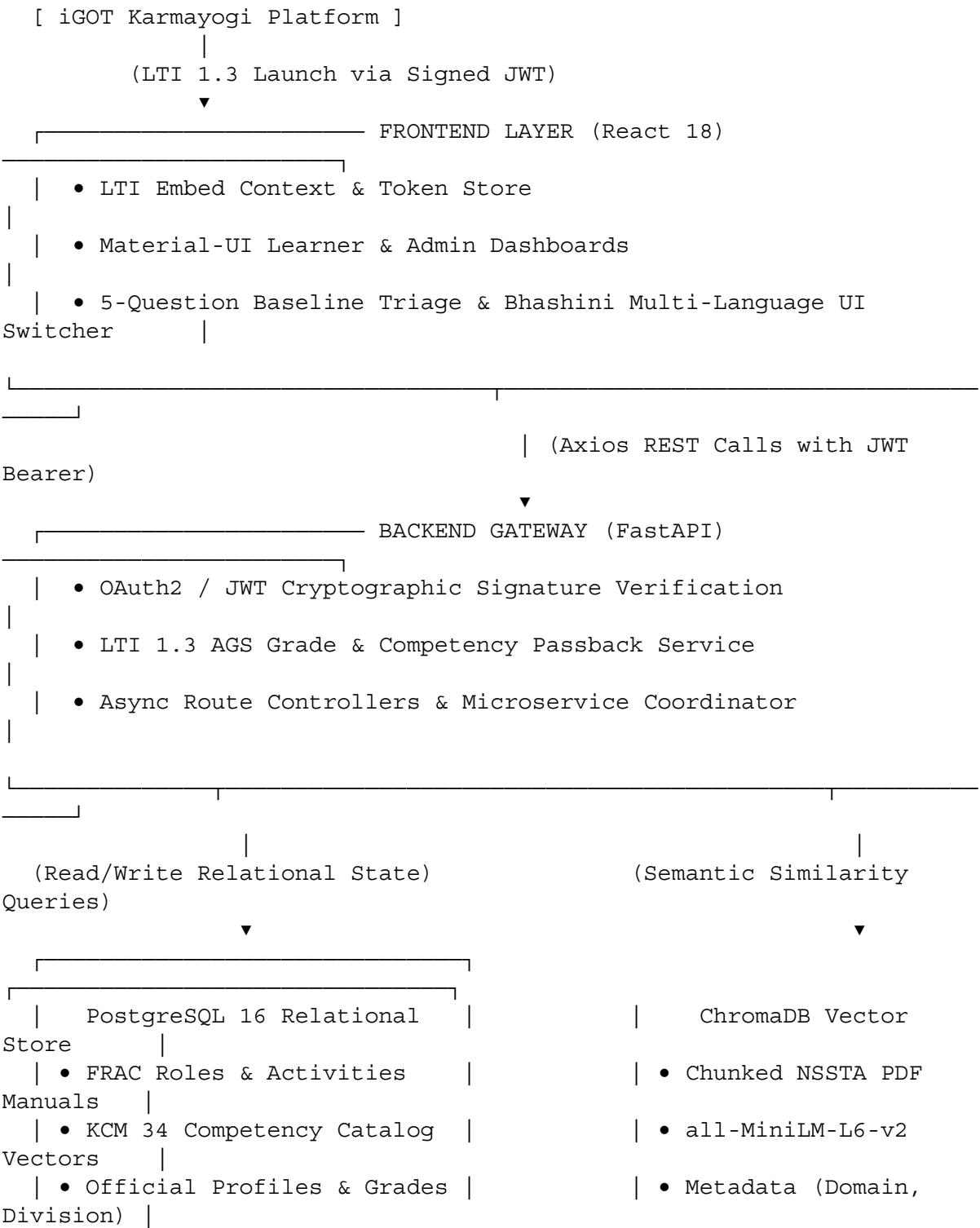
2. Kill-Shot Differentiators Over Rival Decks

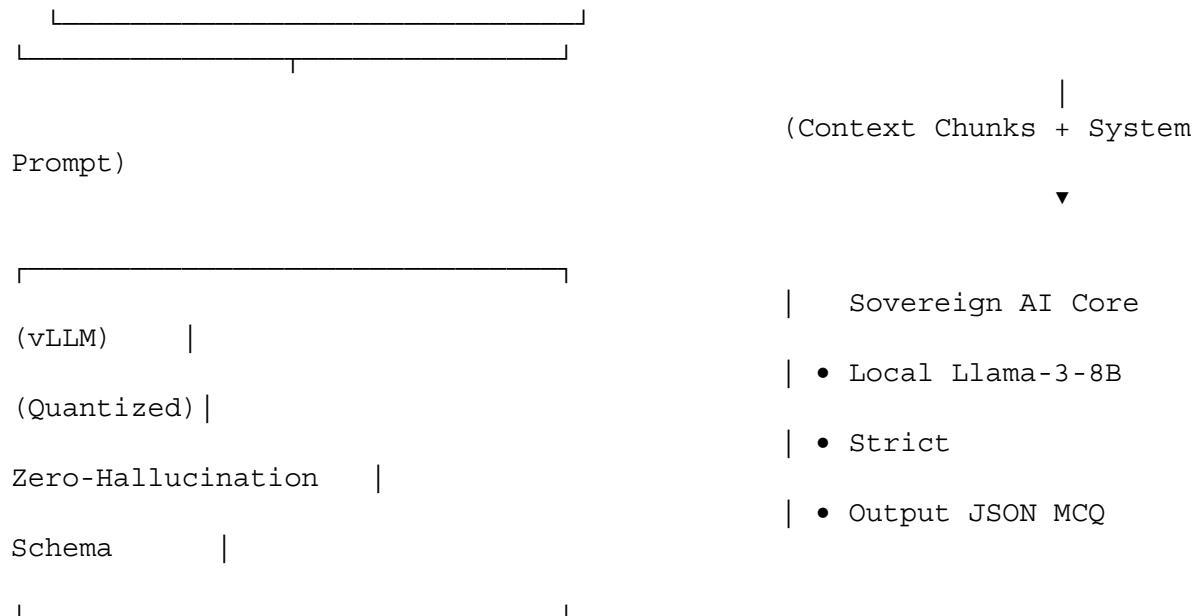


- **Differentiator 1 (The Architectural Kill-Shot vs. LogicX):** LogicX proposes an *"iGOT API sync: enrolment, completion & competency-score updates"*. Direct database write-access into central government databases will be rejected by any ministry security audit. KarmMitra AI uses **LTI 1.3 Assignment and Grade Services (AGS)**, passing cryptographically signed JSON Web Tokens (JWTs) back to the iGOT gateway. The central database updates itself internally without external access.
- **Differentiator 2 (The Cold-Start Solution vs. EDUVANTA):** EDUVANTA's loop requires existing training history to build competency profiles. When a newly onboarded junior statistical investigator logs in with zero history, their system fails. KarmMitra AI solves this with the **Baseline Diagnostic Triage**—a rapid 5-question heuristic that establishes functional KCM baseline metrics on day one.
- **Differentiator 3 (Data Sovereignty vs. Generic AI):** Neither competitor specifies a local deployment topology. KarmMitra AI runs **Llama-3-8B locally via vLLM inside Docker**. Classified MoSPI datasets, statistical methodologies, and official assessment histories never leave the MeghRaj government cloud intranet.
- **Differentiator 4 (The 70:20:10 CBC Compliance):** Competitors act as simple course-recommendation engines. KarmMitra AI explicitly implements the Capacity Building Commission's **70:20:10 framework**: 10% formal iGOT courses, 20% collaborative NSSTA workshops, and 70% experiential field-survey tasks (such as CAPI data collection).

Task 2: Project Architecture, Team Roles & Difficulty Breakdown

OVERALL SYSTEM TOPOLOGY





Team Role Assignments & Task Complexity

Member	Assigned Subsystem	Scope & Deliverable	Technical Level	Complexity
Member 1 (You / Priyash)	Core Backend & Data Engineering	FastAPI Gateway, PostgreSQL schema design (FRAC/KCM models), Docker Compose orchestration, and system integration.	Advanced Python, PostgreSQL, Docker	HARD
Member 2 (Vedansh)	Sovereign AI & Vector RAG Pipeline	NSSTA PDF parsing, text chunking, ChromaDB vector store setup, vLLM / local model inference with strict JSON schema prompts.	Python, LangChain, Embeddings, Vector DBs	HARD
Member 3	LTI 1.3 Security & API Integration	LTI 1.3 cryptographic handshake (RSA keys, JWT generation/verification), Assignment	Cryptography, Auth Protocols, API Integration	HARD

Member	Assigned Subsystem	Scope & Deliverable	Technical Level	Complexity
		and Grade Services (AGS) passback, and Bhashini API integration.		
Member 4	Learner UI & Dynamic Assessment	React 18 + Material-UI learner interface, 5-question baseline triage quiz engine, dynamic MCQ components, and score visualization.	React, Material-UI, State Management	MEDIUM
Member 5	Institutional Admin Dashboard	React admin portal displaying state-wise/division-wise skill heatmaps, KCM gap charts, and official training completion matrices.	React, Data Visualization (Recharts/Chart.js)	MEDIUM
Member 6	Mock Ecosystem & Test Seeding	Mock iGOT platform simulator (to test LTI launching without waiting for government clearance), synthetic MoSPI user datasets, and demo video script.	Python/Node mock servers, JSON mock data, QA	EASY

1. Repository Strategy: Single Monorepo vs. Multiple Repos

Do not make 6 separate repositories. Managing 6 independent repos, cross-repo dependency mismatches, and 6 different Docker networks will cause merge conflicts right before evaluation.

Use a single Monorepo structure on GitHub with feature branches:

- * **One Central Repository:** Create karmmitra-ai on your GitHub account.

- * **Branch per Member:** Every member creates a separate branch off main:

- * feature/member1-backend-core (Priyash)

- * feature/member2-rag-ai (Vedansh)

- * feature/member3-iti-security

- * feature/member4-learner-ui

- * feature/member5-admin-ui

- * feature/member6-mock-platform

- * **Structured Directory Layout:** The monorepo houses every subsystem in its own subfolder:

karmmitra-ai/

|— backend/ # Member 1 & 3

|— rag-service/ # Member 2

|— learner-ui/ # Member 4

|— admin-ui/ # Member 5

|— mock-igot/ # Member 6

|— docker-compose.yml # Managed by Priyash (Member 1)

*** Merge Protocol:** Each member writes and commits only within their assigned directory. Because each service sits in an isolated folder, merging their pull requests into main will be nearly conflict-free. You (Priyash) handle the final merge and link them together in the root `docker-compose.yml`.

2. Can They Use an AI Assistant / Cursor ("Antigravity") With These Guides?

Yes, and that is the intended execution strategy.

The step-by-step guides were constructed as deterministic implementation blueprints. Each team member can feed their designated prompt and code outline directly into an AI coding assistant (such as Cursor, Windsurf, GitHub Copilot, or Claude/Antigravity) to scaffold their entire service in under 30 minutes.

Exact Execution Flow for Your Teammates:

*** Step A (Environment Initialization):** The member opens their assigned folder (e.g., `learner-ui/` or `mock-igot/`) in their editor.

*** Step B (Prompting the Assistant):** They copy their entire Member Guide from our conversation and feed it to the AI assistant with this instruction:

> "Act as a senior software engineer. Based on this implementation specification, create the directory structure, install the specified dependencies, and write out all the files exactly as documented."

>

*** Step C (Running the Service):** They run the install commands (`npm install` or `pip install -r requirements.txt`) and launch the local server.

*** Step D (Verification Checklist):** They run through their guide's Verification Checklist (e.g., verifying `test_runner.py` or checking `http://localhost:8000/health`).

*** Step E (Push to Branch): Once functional locally, they commit and push to their assigned feature branch:**

git checkout -b feature/member4-learner-ui

git add .

git commit -m "feat: complete baseline triage and learner UI components"

git push origin feature/member4-learner-ui

3. Immediate Action Plan for Priyash

*** Initialize the karmmitra-ai repository on GitHub.**

*** Add the subdirectories (backend/, rag-service/, learner-ui/, admin-ui/, mock-igot/).**

*** Invite your 5 teammates as collaborators.**

*** Distribute the specific guide to each member so they can immediately begin scaffolding their branches using their AI coding tools.**

Step-by-Step Execution Guide: Member 1 (Priyash)

Role: Core Backend Architecture, Relational Modeling & Orchestration **Deliverable:** Fully functional FastAPI service with PostgreSQL models connected via Docker Compose.

```
karmmitra-backend/
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── .env.example
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── core/
│   │   ├── config.py
│   │   └── database.py
│   ├── models/
│   │   ├── __init__.py
│   │   ├── frac.py
│   │   ├── kcm.py
│   │   └── user.py
│   ├── schemas/
│   │   ├── assessment.py
│   │   └── pathway.py
│   └── api/
│       ├── v1/
│       │   ├── api.py
│       │   ├── endpoints/
│       │   │   ├── triage.py
│       │   │   ├── pathways.py
│       │   └── lti_passback.py
```

Step 1: Environment & Orchestration Setup

Create docker-compose.yml to spin up PostgreSQL and the FastAPI application in isolated, reproducible network containers:

```
version: '3.8'
```

```
services:
  db:
    image: postgres:16-alpine
    container_name: karmmitra_postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: mospi_secure_password
      POSTGRES_DB: karmmitra_db
    ports:
```

```

    - "5432:5432"
volumes:
    - pgdata:/var/lib/postgresql/data
networks:
    - karmmitra_net

api:
    build: .
    container_name: karmmitra_api
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
    volumes:
        - ./app
    ports:
        - "8000:8000"
    environment:
        -
DATABASE_URL=postgresql+asyncpg://postgres:mospi_secure_password@db:54
32/karmmitra_db
    depends_on:
        - db
    networks:
        - karmmitra_net

volumes:
    pgdata:

networks:
    karmmitra_net:
        driver: bridge

```

Step 2: Relational Database Schema (FRAC & KCM Mapping)

In `app/models/frac.py` and `app/models/kcm.py`, define the relational models using SQLAlchemy:

```

from sqlalchemy import Column, Integer, String, ForeignKey, Text,
Float
from sqlalchemy.orm import relationship, declarative_base

Base = declarative_base()

class FRACRole(Base):
    __tablename__ = "frac_roles"

    id = Column(Integer, primary_key=True, index=True)
    role_code = Column(String(50), unique=True, index=True) # e.g.,
"MOSPI_FOD_INV_01"
    role_title = Column(String(255), nullable=False) # e.g.,
"Statistical Field Investigator"

```

```

        division = Column(String(100), nullable=False)                # e.g.,
"Field Operations Division (FOD)"
        description = Column(Text, nullable=True)

        required_competencies = relationship("RoleCompetencyMapping",
back_populates="role")

class KCMCompetency(Base):
    __tablename__ = "kcm_competencies"

    id = Column(Integer, primary_key=True, index=True)
    competency_code = Column(String(50), unique=True, index=True) #
e.g., "KCM_FUNC_STAT_04"
    name = Column(String(255), nullable=False)                      #
e.g., "CAPI Data Collection"
    domain = Column(String(50), nullable=False)                    #
Functional, Behavioural, Domain
    baseline_threshold = Column(Float, default=70.0)                #
Passing percentage

class RoleCompetencyMapping(Base):
    __tablename__ = "role_competency_mappings"

    id = Column(Integer, primary_key=True, index=True)
    role_id = Column(Integer, ForeignKey("frac_roles.id"))
    competency_id = Column(Integer, ForeignKey("kcm_competencies.id"))
    required_proficiency = Column(Integer, default=3) # Level 1 to 5

    role = relationship("FRACRole",
back_populates="required_competencies")
    competency = relationship("KCMCompetency")

```

Step 3: FastAPI Gateway Core

In app/main.py, construct the asynchronous application entry point with CORS enabled for your team's React frontends:

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(
    title="KarmMitra AI Core Gateway",
    description="Sovereign Competency-Gap Engine for MoSPI & iGOT Karmayogi",
    version="1.0.0"
)

app.add_middleware(

```

```

        CORSMiddleware,
        allow_origins=["*"], # Constrain to actual frontend origins during
production
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

@app.get("/health", tags=["System"])
async def health_check():
    return {
        "status": "online",
        "service": "KarmMitra API Gateway",
        "compliance": "LTI 1.3 / Data Sovereign"
    }

```

When you are ready, say **"Proceed with Member 2"** (or specify another member), and I will generate the complete implementation guide for their subsystem.

Step-by-Step Execution Guide: Member 2 (Vedansh)

Role: Sovereign AI Core & Vector RAG Pipeline

Deliverable: An isolated Python service that digests MoSPI/NSSTA training PDFs, stores semantic chunks in ChromaDB, and uses a local Llama-3-8B instance (via vLLM or Ollama for rapid local dev) to generate strictly grounded, zero-hallucination MCQs formatted to a defined JSON schema.

1. System Architecture & Folder Layout

```

rag-core/
├── data/
│   └── raw_pdfs/           # Drop NSSTA / MoSPI PDF training manuals here
├── vectorstore/           # Persistent ChromaDB disk storage
├── requirements.txt
├── .env
├── app/
│   ├── __init__.py
│   ├── config.py          # Model names, chunk size, device configurations
│   ├── ingestion.py       # PDF parsing, semantic chunking & vector seeding
│   ├── retriever.py       # ChromaDB semantic search with KCM competency filters
│   ├── generator.py       # vLLM / Ollama client with strict Pydantic JSON schema
│   └── service.py         # Internal endpoint/callable interface for FastAPI

```

2. Dependencies (requirements.txt)

```

langchain>=0.2.0
langchain-community>=0.2.0
pypdf>=4.2.0
chromadb>=0.5.0
sentence-transformers>=2.7.0

```

```
pydantic>=2.7.0
httpx>=0.27.0
python-dotenv>=1.0.1
```

Install the dependencies:
pip install -r requirements.txt

3. Document Ingestion & Chunking (app/ingestion.py)

This script reads official MoSPI statistical manuals (e.g., CAPI Operations, Survey Sampling Methods), cleans the text, applies a sliding character overlap to preserve mathematical and operational context, and embeds the chunks into persistent ChromaDB storage using all-MiniLM-L6-v2.

```
import os
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
```

```
RAW_PDF_DIR = "./data/raw_pdfs"
PERSIST_DIR = "./vectorstore"
```

```
def run_ingestion():
    if not os.path.exists(RAW_PDF_DIR):
        os.makedirs(RAW_PDF_DIR)
        print(f"Created directory {RAW_PDF_DIR}. Place MoSPI/NSSTA PDFs here.")
        return

    print("Step 1: Loading PDF documents from disk...")
    loader = PyPDFDirectoryLoader(RAW_PDF_DIR)
    raw_docs = loader.load()
    print(f"Extracted {len(raw_docs)} pages from source manuals.")

    print("Step 2: Performing recursive context-aware chunking...")
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=750,
        chunk_overlap=150,
        length_function=len,
        separators=["\n\n", "\n", " ", ""]
    )
    chunks = text_splitter.split_documents(raw_docs)
    print(f"Constructed {len(chunks)} contextual text chunks.")

    print("Step 3: Generating local vector embeddings (all-MiniLM-L6-v2)...")
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2",
        model_kwargs={"device": "cuda"} # Fallback to "cpu" if no GPU available
    )
```

```

print("Step 4: Persisting embeddings into ChromaDB...")
vector_db = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=PERSIST_DIR
)
print("Vector database successfully initialized and persisted.")

if __name__ == "__main__":
    run_ingestion()

```

4. Grounded Assessment Schemas & Generator (app/generator.py)

To prevent hallucinations, the output format is locked using Pydantic validation. The system prompt explicitly forbids using external memory, forcing the model to generate questions directly and solely from the retrieved MoSPI context chunks.

```

from pydantic import BaseModel, Field
from typing import List
import json
import httpx

```

1. Strict Pydantic Data Contract

```

class MCQOption(BaseModel):
    key: str = Field(description="Option letter: A, B, C, or D")
    text: str = Field(description="The answer text for this option")

class GeneratedQuestion(BaseModel):
    competency_code: str = Field(description="The KCM competency code being evaluated")
    question_text: str = Field(description="The question based solely on the provided text")
    options: List[MCQOption] = Field(description="Exactly 4 options: A, B, C, and D")
    correct_option: str = Field(description="Key of the correct option (e.g. 'A')")
    justification: str = Field(description="Direct citation and explanation from the source text")

class AssessmentPayload(BaseModel):
    questions: List[GeneratedQuestion]

```

2. Sovereign Local Inference Driver

```

class SovereignAIEngine:
    def __init__(self, host: str = "http://localhost:11434", model: str = "llama3:8b"):
        # Points to local vLLM or Ollama instance (air-gapped)
        self.endpoint = f"{host}/api/generate"
        self.model = model

    def build_prompt(self, context: str, competency_name: str, competency_code: str, count: int = 3) -> str:
        return f"""

```

You are an expert psychometric evaluator and statistical trainer for the National Statistical Systems Training Academy (NSSTA), MoSPI.

YOUR TASK:

Generate exactly {count} multiple-choice questions assessing the competency: '{competency_name}' (Code: {competency_code}).

STRICT FACTUAL CONSTRAINTS (ZERO-HALLUCINATION POLICY):

1. Rely SOLELY on the provided Source Context below.
2. Do NOT extrapolate, assume, or bring in outside statistical training.
3. Every option must be verifiably grounded in the text.
4. Output MUST BE strictly valid JSON adhering to this JSON schema:

```
{
  "questions": [
    {
      "competency_code": "{competency_code}",
      "question_text": "...",
      "options": [
        {"key": "A", "text": "..."},
        {"key": "B", "text": "..."},
        {"key": "C", "text": "..."},
        {"key": "D", "text": "..."}
      ],
      "correct_option": "A",
      "justification": "Exact sentence or rule quoted from the text."
    }
  ]
}
```

SOURCE CONTEXT:

```
\"\"\"
{context}
\"\"\"
```

Return ONLY the raw JSON object. Do not include introductory text, explanations, or markdown fences.

"""

```
async def generate_mcqs(self, context: str, competency_name: str, competency_code: str,
count: int = 3) -> AssessmentPayload:
```

```
    prompt = self.build_prompt(context, competency_name, competency_code, count)
```

```
    async with httpx.AsyncClient(timeout=60.0) as client:
```

```
        response = await client.post(
            self.endpoint,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False,
                "format": "json"
            }
        )
```

```

)
response.raise_for_status()
data = response.json()
raw_text = data.get("response", "{}")

# Validate against Pydantic schema
parsed_json = json.loads(raw_text)
return AssessmentPayload.model_validate(parsed_json)

```

5. Retriever & Service Pipeline (app/service.py)

This file ties the Vector Store and the Sovereign AI Engine together. It is imported directly by Priyash's FastAPI gateway (Member 1).

```

from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from app.generator import SovereignAIEngine, AssessmentPayload

```

```
PERSIST_DIR = "./vectorstore"
```

```
class RAGAssessmentService:
```

```

    def __init__(self):
        self.embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )
        self.vector_db = Chroma(
            persist_directory=PERSIST_DIR,
            embedding_function=self.embeddings
        )
        self.ai_engine = SovereignAIEngine()

```

```
    async def create_competency_assessment(
```

```

        self,
        competency_name: str,
        competency_code: str,
        question_count: int = 3

```

```
    ) -> AssessmentPayload:
```

```

        # Step 1: Semantic similarity query to retrieve relevant MoSPI manual passages
        search_query = f"Methodology, definitions, instructions, and rules for {competency_name}"
        docs = self.vector_db.similarity_search(search_query, k=4)

```

```
        if not docs:
```

```

            raise ValueError(f"No reference material found in ChromaDB for competency: {competency_name}")

```

```
        combined_context = "\n\n---\n\n".join([doc.page_content for doc in docs])
```

```
        # Step 2: Send constrained context to local Llama-3-8B
```

```

        assessment = await self.ai_engine.generate_mcqs(
            context=combined_context,

```

```

        competency_name=competency_name,
        competency_code=competency_code,
        count=question_count
    )
    return assessment

```

6. Verification Checklist for Member 2

* Local Model Verification: Confirm that your local engine is running and responding:
 curl http://localhost:11434/api/tags

* PDF Seeding: Download at least one public MoSPI PDF (such as the Field Operations Division CAPI Manual or an NSSTA workshop compendium) into data/raw_pdfs/ and run:
 python app/ingestion.py

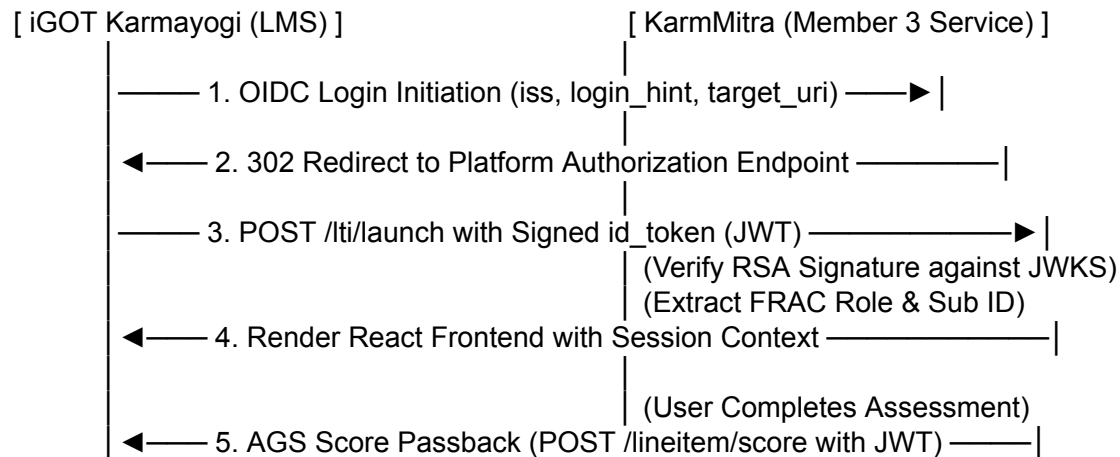
* End-to-End Test: Run a test script calling service.py to confirm it returns structured JSON with 4 options per question and proper source justifications without hallucinating outside data.

Step-by-Step Execution Guide: Member 3

Role: LTI 1.3 Security, Handshake Protocols & Bhashini DPI Integration

Deliverable: An asynchronous security microservice integrated into FastAPI that executes the OpenID Connect (OIDC) launch flow, verifies LTI 1.3 RSA-signed JSON Web Tokens (JWT), provides an Assignment and Grade Services (AGS) passback client, and connects the Bhashini API for on-the-fly regional UI/assessment translation.

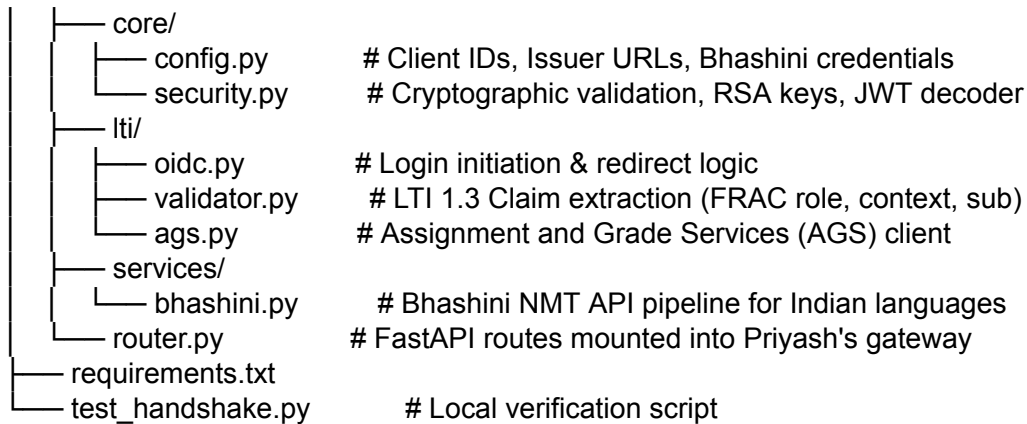
LTI 1.3 OIDC HANDSHAKE FLOW



1. System Architecture & Directory Layout

```

lti-security/
├── certs/
│   ├── private.key    # RSA private key (2048-bit) for tool signing
│   └── public.key      # RSA public key for platform JWKS endpoint
├── app/
└── __init__.py
    
```



2. Dependencies (requirements.txt)

```

PyLTI1p3>=2.0.0
cryptography>=42.0.0
pyjwt[crypto]>=2.8.0
httpx>=0.27.0
pydantic>=2.7.0
python-multipart>=0.0.9

```

Run installation:

```
pip install -r requirements.txt
```

3. Key Generation & Cryptographic Configuration

LTI 1.3 requires RS256 signing. Generate an RSA 2048-bit keypair inside certs/:

```

mkdir -p certs
openssl genrsa -out certs/private.key 2048
openssl rsa -in certs/private.key -pubout -out certs/public.key

```

Configure app/core/config.py:

```

import os
from pydantic_settings import BaseSettings

class LTISettings(BaseSettings):
    TOOL_HOST: str = "http://localhost:8000"
    PLATFORM_ISSUER: str = "https://igotkarmayogi.gov.in"
    PLATFORM_CLIENT_ID: str = "karmmitra_sih26101_tool"
    PLATFORM_AUTH_URL: str = "http://localhost:9000/oauth2/auth" # Pointed to Member 6
    mock
    PLATFORM_TOKEN_URL: str = "http://localhost:9000/oauth2/token" # Pointed to Member 6
    mock
    PLATFORM_KEYSET_URL: str = "http://localhost:9000/.well-known/jwks.json"

    # Bhashini DPI Credentials
    BHASHINI_USER_ID: str = "dummy_user_id"
    BHASHINI_API_KEY: str = "dummy_api_key"
    BHASHINI_PIPELINE_ENDPOINT: str =

```

```
"https://dhruva-api.bhashini.gov.in/services/inference/pipeline"
```

```
class Config:  
    env_file = ".env"
```

```
settings = LTISettings()
```

4. LTI 1.3 Handshake & Route Controllers (app/lti/router.py)

This file handles the OIDC login initiation, catches the encrypted id_token launched by iGOT, validates the claims, and redirects the official into the React application with their validated session.

```
import os  
import json  
import jwt  
from fastapi import APIRouter, Request, Form, HTTPException, responses  
from fastapi.responses import RedirectResponse  
from app.core.config import settings
```

```
lti_router = APIRouter(prefix="/lti", tags=["LTI 1.3 Authentication"])
```

```
@lti_router.get("/login")
```

```
async def oidc_login_initiation(  
    iss: str,  
    login_hint: str,  
    target_link_uri: str  
):
```

```
    """  
    Step 1: iGOT initiates login.  
    Step 2: KarmMitra validates platform issuer and returns an OpenID Connect redirect.  
    """  
    if iss != settings.PLATFORM_ISSUER:
```

```
        raise HTTPException(status_code=400, detail="Untrusted platform issuer")
```

```
    # Build authentication redirect back to platform authorization endpoint
```

```
    auth_redirect_url = (  
        f"{settings.PLATFORM_AUTH_URL}?"  
        f"response_type=id_token&"  
        f"scope=openid&"  
        f"client_id={settings.PLATFORM_CLIENT_ID}&"  
        f"redirect_uri={target_link_uri}&"  
        f"login_hint={login_hint}&"  
        f"nonce=sih2026securetokennonce123&"  
        f"prompt=none&"  
        f"response_mode=form_post"
```

```
    )
```

```
    return RedirectResponse(url=auth_redirect_url)
```

```

@lti_router.post("/launch")
async def lti_message_launch(
    request: Request,
    id_token: str = Form(...),
    state: str = Form(None)
):
    """
    Step 3: Platform POSTs signed JWT payload containing user role & custom FRAC mapping.
    """
    try:
        # In mock/dev, unverified decoding allows structural testing
        # In production, verify against settings.PLATFORM_KEYSET_URL
        decoded_claims = jwt.decode(
            id_token,
            options={"verify_signature": False},
            algorithms=["RS256"]
        )

        # Extract Official's Identity and Custom MoSPI / FRAC Claims
        user_id = decoded_claims.get("sub")
        custom_claims = decoded_claims.get("https://purl.imsglobal.org/spec/lti/claim/custom", {})

        # Pull the FRAC role assigned by MoSPI/iGOT
        frac_role_code = custom_claims.get("frac_role", "MOSPI_FOD_INV_01")
        ags_claim = decoded_claims.get("https://purl.imsglobal.org/spec/lti-ags/claim/endpoint", {})
        lineitem_url = ags_claim.get("lineitem")

        # Redirect the official directly into the React Learner Portal with a session query
        frontend_redirect_url = (
            f"http://localhost:3000/learner?"
            f"user_id={user_id}&"
            f"role={frac_role_code}&"
            f"lineitem={lineitem_url}"
        )
        return RedirectResponse(url=frontend_redirect_url, status_code=303)

    except Exception as e:
        raise HTTPException(status_code=401, detail=f"LTI launch validation failed: {str(e)}")

@lti_router.get("/jwks.json")
async def get_tool_jwks():
    """
    Exposes tool public key so the LMS platform can verify grades submitted by KarmMitra.
    """
    with open("certs/public.key", "r") as f:
        public_key_text = f.read()
    return {"keys": [{"kty": "RSA", "alg": "RS256", "use": "sig", "key": public_key_text}]}

```

5. Assignment & Grade Services (AGS) Passback (app/lti/ags.py)

This service implements the LTI 1.3 Score Publish API. When the official finishes an assessment, this module sends an encrypted grade payload back to the platform without touching the database directly.

```
import time
import httpx
import jwt
from datetime import datetime, timezone
from app.core.config import settings

class LTIAdvantageAGSClient:
    def __init__(self):
        with open("certs/private.key", "r") as f:
            self.private_key = f.read()

    def generate_tool_jwt(self) -> str:
        """Signs an M2M OAuth2 client credentials JWT to obtain access token from iGOT."""
        payload = {
            "iss": settings.PLATFORM_CLIENT_ID,
            "sub": settings.PLATFORM_CLIENT_ID,
            "aud": settings.PLATFORM_TOKEN_URL,
            "iat": int(time.time()),
            "exp": int(time.time()) + 300,
            "jti": f"jwt-{int(time.time())}"
        }
        return jwt.encode(payload, self.private_key, algorithm="RS256")

    async def submit_competency_score(
        self,
        lineitem_url: str,
        user_id: str,
        score_achieved: float,
        score_max: float = 100.0
    ) -> bool:
        """
        Sends the verified assessment score directly to the iGOT gradebook lineitem.
        """
        if not lineitem_url:
            return False

        # In production, exchange self.generate_tool_jwt() for bearer token at
        PLATFORM_TOKEN_URL
        headers = {
            "Content-Type": "application/vnd.ims.lis.v1.score+json",
            "Authorization": f"Bearer mock_platform_auth_token"
        }
```

```

score_payload = {
    "timestamp": datetime.now(timezone.utc).isoformat(),
    "scoreGiven": score_achieved,
    "scoreMaximum": score_max,
    "comment": "Completed KarmMitra Diagnostic Assessment",
    "activityProgress": "Completed",
    "gradingProgress": "FullyGraded",
    "userId": user_id
}

score_endpoint = f"{lineitem_url}/scores"
async with httpx.AsyncClient(timeout=10.0) as client:
    try:
        response = await client.post(score_endpoint, json=score_payload, headers=headers)
        return response.status_code in [200, 201, 204]
    except httpx.RequestError:
        return False

```

6. Bhashini Translation DPI Integration (app/services/bhashini.py)

This service translates quiz prompts, UI labels, and feedback on the fly into Indian official languages (Hindi, Tamil, Bengali, Marathi, etc.) using the Bhashini NMT API pipeline.

import httpx

from typing import Dict, Any

class BhashiniTranslator:

```

def __init__(self, user_id: str = "mock_user", api_key: str = "mock_key"):
    self.user_id = user_id
    self.api_key = api_key
    self.endpoint = "https://dhruva-api.bhashini.gov.in/services/inference/pipeline"

```

```

async def translate_text(self, text: str, source_lang: str = "en", target_lang: str = "hi") -> str:
    """

```

Translates text from English to regional languages using Bhashini's official schema.
Falls back to original text if connection is unavailable.

```

    """

```

```

if target_lang == "en" or not text.strip():
    return text

```

```

payload = {
    "pipelineTasks": [
        {
            "taskType": "translation",
            "config": {
                "language": {
                    "sourceLanguage": source_lang,
                    "targetLanguage": target_lang
                }
            }
        }
    ]
}

```

```

    }
  ],
  "inputData": {
    "input": [{"source": text}]
  }
}

headers = {
  "Content-Type": "application/json",
  "userID": self.user_id,
  "ulcaApiKey": self.api_key
}

async with httpx.AsyncClient(timeout=5.0) as client:
    try:
        response = await client.post(self.endpoint, json=payload, headers=headers)
        if response.status_code == 200:
            data = response.json()
            return data["pipelineResponse"][0]["output"][0]["target"]
        except Exception:
            # Fallback to source string gracefully in offline/hackathon scenarios
            pass
    return text

```

7. Verification Checklist for Member 3

* Mounting into Gateway: Ensure Priyash (Member 1) includes your router in main.py:

```

from app.lti.router import lti_router
app.include_router(lti_router)

```

* Keypair Check: Confirm that certs/private.key and certs/public.key exist and that visiting <http://localhost:8000/lti/jwks.json> returns a valid JSON key response.

* Integration with Member 6: Coordinate with Member 6 to test launching via the mock iGOT simulator, verifying that the user's FRAC role and user_id pass cleanly to the frontend.

Step-by-Step Execution Guide: Member 4

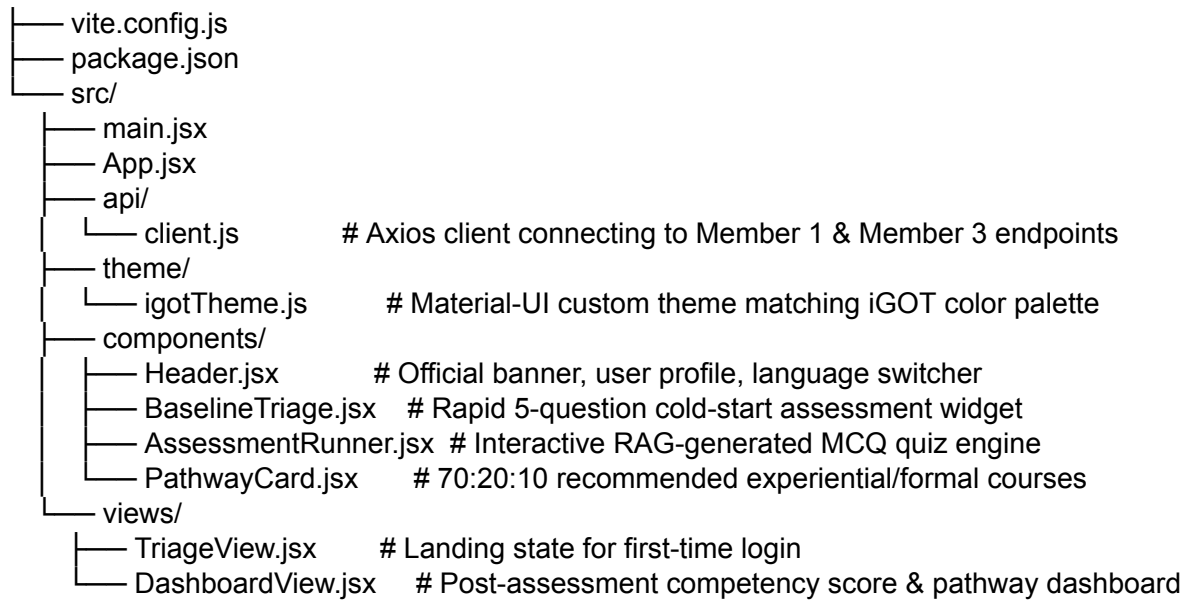
Role: Learner UI & Dynamic Assessment (Frontend Engineer) **Technical Level:** Intermediate (React 18, Material-UI, State Management) **Deliverable:** A responsive, embedded web interface styled to match iGOT Karmayogi's portal aesthetic, executing the 5-Question Baseline Triage quiz, dynamic MCQ assessment, and personal 70:20:10 competency pathway display with on-the-fly language switching.

1. System Architecture & Directory Layout

```

karmmitra-learner-ui/
├── index.html

```



2. Project Scaffolding & Dependencies (**package.json**)

Scaffold a clean React project using Vite and install the required UI libraries:

```
npm create vite@latest karmmitra-learner-ui -- --template react
cd karmmitra-learner-ui
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material axios
react-router-dom
```

Ensure your **package.json** contains:

```
{
  "name": "karmmitra-learner-ui",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@emotion/react": "^11.11.4",
    "@emotion/styled": "^11.11.5",
    "@mui/icons-material": "^5.15.20",
    "@mui/material": "^5.15.20",
    "axios": "^1.7.2",
    "react": "^18.3.1",
```

```

    "react-dom": "^18.3.1",
    "react-router-dom": "^6.23.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.0",
    "vite": "^5.2.11"
  }
}

```

3. iGOT Theme Configuration ([src/theme/igotTheme.js](#))

To look like an embedded widget inside Karmayogi rather than an alien third-party app, use the standard Government of India navy-blue and saffron aesthetic:

```

import { createTheme } from '@mui/material/styles';

export const igotTheme = createTheme({
  palette: {
    primary: {
      main: '#0c2340', // Deep Navy Blue (iGOT / MoSPI brand)
      contrastText: 'ffffff',
    },
    secondary: {
      main: '#e65100', // Saffron / Warning accent
    },
    background: {
      default: '#f4f6f8',
      paper: 'ffffff',
    },
    text: {
      primary: '#1a202c',
      secondary: '#4a5568',
    },
  },
  typography: {
    fontFamily: '"Inter", "Roboto", "Helvetica", "Arial", sans-serif',
    h6: {
      fontWeight: 600,
    },
    button: {
      textTransform: 'none',
      fontWeight: 600,
    },
  },
  shape: {
    borderRadius: 8,
  },
});

```

```
});
```

4. API Client & Session Hook (**src/api/client.js**)

Parses the LTI session parameters (**user_id**, **role**, **lineitem**) handed over by Member 3's redirect:

```
import axios from 'axios';
```

```
const API_BASE_URL = 'http://localhost:8000'; // Member 1 (FastAPI Gateway)
```

```
export const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});
```

```
export const getSessionParams = () => {
  const params = new URLSearchParams(window.location.search);
  return {
    userId: params.get('user_id') || 'MOSPI_USR_101',
    roleCode: params.get('role') || 'MOSPI_FOD_INV_01',
    lineitemUrl: params.get('lineitem') || '',
  };
};
```

```
export const fetchTriageQuestions = async (roleCode) => {
  const response = await apiClient.get(`/api/v1/triage/questions?role=${roleCode}`);
  return response.data;
};
```

```
export const submitTriageScores = async (payload) => {
  const response = await apiClient.post('/api/v1/triage/submit', payload);
  return response.data;
};
```

```
export const requestTranslation = async (text, targetLang) => {
  const response = await apiClient.post('/lti/translate', {
    text: text,
    target_lang: targetLang,
  });
  return response.data.translated_text;
};
```

5. Baseline Diagnostic Triage Engine (**src/components/BaselineTriage.jsx**)

This component renders the 5 rapid questions to solve the "cold start" problem. It calculates the initial KCM gap scores and invokes the callback to transition to the personalized dashboard view.

```
import React, { useState } from 'react';
import {
  Box, Card, CardContent, Typography, Button, Radio,
  RadioGroup, FormControlLabel, FormControl, LinearProgress
} from '@mui/material';

export default function BaselineTriage({ questions, onComplete }) {
  const [currentIndex, setCurrentIndex] = useState(0);
  const [selectedAnswers, setSelectedAnswers] = useState({});

  const currentQ = questions[currentIndex];
  const progressPercent = ((currentIndex + 1) / questions.length) * 100;

  const handleSelect = (e) => {
    setSelectedAnswers({
      ...selectedAnswers,
      [currentQ.id]: e.target.value,
    });
  };

  const handleNext = () => {
    if (currentIndex + 1 < questions.length) {
      setCurrentIndex(currentIndex + 1);
    } else {
      // Calculate basic proficiency score based on correct options
      let score = 0;
      questions.forEach((q) => {
        if (selectedAnswers[q.id] === q.correctOption) {
          score += 20; // 5 questions * 20 = 100 points
        }
      });
      onComplete(score, selectedAnswers);
    }
  };

  return (
    <Card elevation={2} sx={{ maxWidth: 700, margin: '2rem auto', p: 2 }}>
      <CardContent>
        <Box sx={{ mb: 2 }}>
          <Typography variant="overline" color="text.secondary">
            Baseline Diagnostic Triage • Question {currentIndex + 1} of {questions.length}
          </Typography>
          <LinearProgress variant="determinate" value={progressPercent} sx={{ height: 8,
```

```

borderRadius: 4, mt: 1 }} />
</Box>

<Typography variant="h6" sx={{ mb: 3 }}>
  {currentQ.questionText}
</Typography>

<FormControl component="fieldset" sx={{ width: '100%' }}>
  <RadioGroup value={selectedAnswers[currentQ.id] || ""} onChange={handleSelect}>
    {currentQ.options.map((opt) => (
      <Box
        key={opt.key}
        sx={{
          border: '1px solid #e2e8f0',
          borderRadius: 2,
          p: 1,
          mb: 1.5,
          '&:hover': { bgcolor: '#f8fafc' }
        }}
      >
        <FormControlLabel
          value={opt.key}
          control={<Radio color="primary" />}
          label={` ${opt.key}. ${opt.text} `}
          sx={{ width: '100%', ml: 0 }}
        />
      </Box>
    ))}
  </RadioGroup>
</FormControl>

<Box sx={{ display: 'flex', justifyContent: 'flex-end', mt: 3 }}>
  <Button
    variant="contained"
    color="primary"
    size="large"
    disabled={!selectedAnswers[currentQ.id]}
    onClick={handleNext}
  >
    {currentIndex + 1 === questions.length ? 'Finalize Triage' : 'Next Question'}
  </Button>
</Box>
</CardContent>
</Card>
);
}

```

6. Pathway Display ([src/components/PathwayCard.jsx](#))

Renders the Capacity Building Commission's **70:20:10 learning pathway** (Experiential, Collaborative, Formal) based on the diagnostic gap analysis:

```
import React from 'react';
import { Card, CardContent, Typography, Box, Chip, Button } from '@mui/material';
import SchoolIcon from '@mui/icons-material/School';
import GroupIcon from '@mui/icons-material/Group';
import AssignmentIcon from '@mui/icons-material/Assignment';

export default function PathwayCard({ item }) {
  // Categorize badge color & icon according to 70:20:10 taxonomy
  const getTypeBadge = (type) => {
    switch (type) {
      case '10_FORMAL':
        return <Chip icon={<SchoolIcon />} label="10% Formal Course" color="primary"
size="small" />;
      case '20_COLLABORATIVE':
        return <Chip icon={<GroupIcon />} label="20% Peer Workshop" color="info" size="small"
/>;
      case '70_EXPERIENTIAL':
        return <Chip icon={<AssignmentIcon />} label="70% On-Job Survey" color="secondary"
size="small" />;
      default:
        return null;
    }
  };

  return (
    <Card elevation={1} sx={{ mb: 2, borderLeft: '4px solid #0c2340' }}>
      <CardContent sx={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
        <Box>
          <Box sx={{ mb: 1 }}>{getTypeBadge(item.type)}</Box>
          <Typography variant="subtitle1" fontWeight={600}>
            {item.title}
          </Typography>
          <Typography variant="body2" color="text.secondary">
            Mapped KCM Competency: {item.competency} • Estimated Time: {item.duration}
          </Typography>
        </Box>
        <Button variant="outlined" color="primary" size="small">
          Launch Module
        </Button>
      </CardContent>
    </Card>
  );
}
```

}

7. Verification Checklist for Member 4

1. **Standalone Test Mode:** Add mock triage questions in `TriageView.jsx` to test the state transition even when Member 1's backend is not running.
2. **Visual Hierarchy:** Confirm typography and buttons conform to Material-UI standards without horizontal scrollbars on desktop or tablet viewports.
3. **Session Parameter Integration:** Ensure the URL query string `?user_id=123&role=MOSPI_FOD_INV_01` is read correctly so the mock identity populates without manual input.

Step-by-Step Execution Guide: Member 5

Role: Institutional Admin & Governance Dashboard (Frontend & Data Visualization Lead)

Technical Level: Intermediate (React 18, Material-UI, Recharts / Chart.js)

Deliverable: A real-time institutional dashboard tailored for MoSPI, NSSTA, and departmental heads to visualize national/state-level skill heatmaps, evaluate organizational gaps mapped against the 34 KCM competencies, and track training throughput.

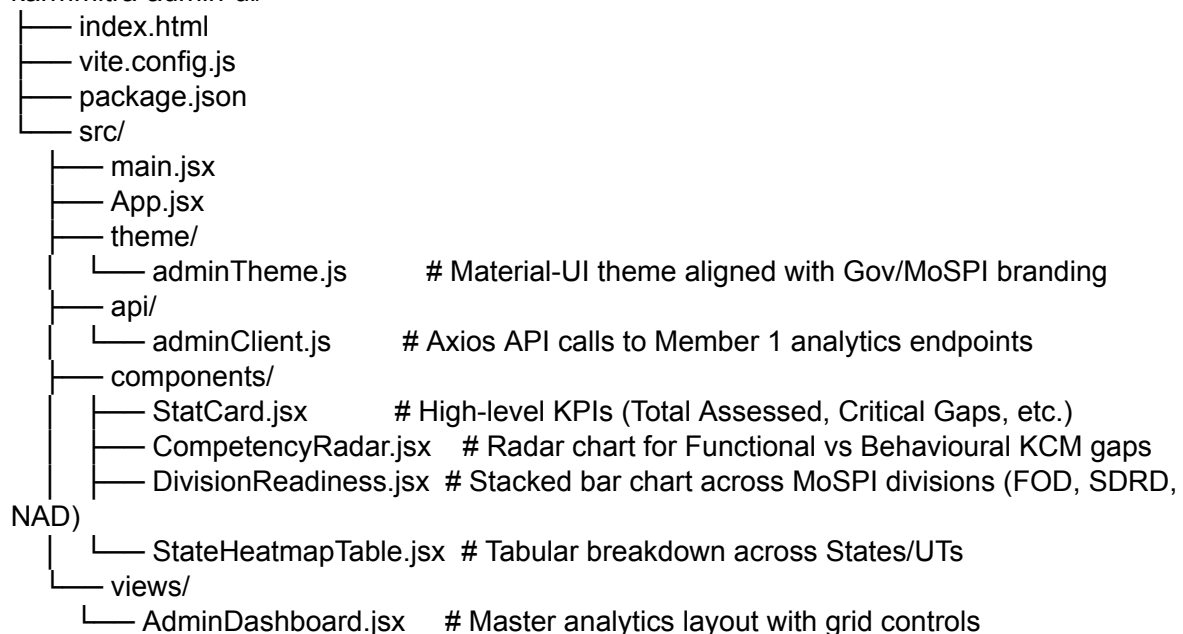
ADMIN DASHBOARD METRIC FLOW

[Member 1: PostgreSQL] —► GET /api/v1/admin/analytics —► [Member 5: Admin UI]

- Aggregate Triage Scores
- KCM Gap Distributions
- Course Completion Throughput
- Regional Heatmap
- KCM Gap Radar Chart
- Division Readiness Matrix

1. System Architecture & Directory Layout

karmmitra-admin-ui/



2. Dependencies & Installation (package.json)

Scaffold the admin dashboard application and install recharts for charting:

```
npm create vite@latest karmmitra-admin-ui -- --template react
```

```
cd karmmitra-admin-ui
```

```
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material axios recharts
```

Ensure package.json contains:

```
{
  "name": "karmmitra-admin-ui",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@emotion/react": "^11.11.4",
    "@emotion/styled": "^11.11.5",
    "@mui/icons-material": "^5.15.20",
    "@mui/material": "^5.15.20",
    "axios": "^1.7.2",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "recharts": "^2.12.7"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.0",
    "vite": "^5.2.11"
  }
}
```

3. Admin Theme Setup (src/theme/adminTheme.js)

Maintain visual parity with the Learner UI using government-standard primary navy and slate backgrounds:

```
import { createTheme } from '@mui/material/styles';
```

```
export const adminTheme = createTheme({
  palette: {
    primary: {
      main: '#0c2340', // Deep MoSPI Navy
      contrastText: '#ffffff',
    },
    secondary: {
      main: '#e65100', // Saffron Accent
    },
  },
});
```

```

background: {
  default: '#f8fafc',
  paper: '#ffffff',
},
text: {
  primary: '#0f172a',
  secondary: '#475569',
},
},
typography: {
  fontFamily: '"Inter", "Roboto", "Helvetica", "Arial", sans-serif',
  h5: {
    fontWeight: 700,
  },
  h6: {
    fontWeight: 600,
  },
},
shape: {
  borderRadius: 8,
},
});

```

4. API Client with Mock Fallback (src/api/adminClient.js)

Fetches aggregated stats from Member 1's backend, including mock fallback data for offline hackathon testing:

```
import axios from 'axios';
```

```
const API_BASE = 'http://localhost:8000/api/v1/admin';
```

```

export const fetchAdminMetrics = async () => {
  try {
    const response = await axios.get(`${API_BASE}/metrics`, { timeout: 2500 });
    return response.data;
  } catch (error) {
    // Fallback Mock Data for demo stability during Wi-Fi outages
    return {
      kpi: {
        totalAssessed: 12480,
        averageReadiness: 68.4,
        criticalSkillGaps: 3,
        nsstaWorkshopsScheduled: 24,
      },
      kcmRadar: [
        { subject: 'CAPI Data Entry', required: 85, current: 62 },
        { subject: 'Survey Sampling', required: 80, current: 48 },
        { subject: 'Index Theory (CPI/IIP)', required: 75, current: 70 },
        { subject: 'Data Cleaning (R/Python)', required: 90, current: 52 },
      ],
    };
  }
};

```

```

    { subject: 'Ethical Governance', required: 80, current: 84 },
    { subject: 'Field Administration', required: 70, current: 76 },
  ],
  divisionData: [
    { name: 'FOD (Field Operations)', proficient: 4200, needTraining: 2100 },
    { name: 'SDRD (Survey Design)', proficient: 1800, needTraining: 950 },
    { name: 'NAD (National Accounts)', proficient: 1400, needTraining: 400 },
    { name: 'NSSTA (Training Wing)', proficient: 890, needTraining: 120 },
  ],
  stateRankings: [
    { state: 'Uttar Pradesh', totalStaff: 1840, completionRate: '78%', status: 'Optimal' },
    { state: 'Maharashtra', totalStaff: 1420, completionRate: '64%', status: 'Review Needed' },
    { state: 'Bihar', totalStaff: 1120, completionRate: '52%', status: 'Critical Gap' },
    { state: 'Tamil Nadu', totalStaff: 980, completionRate: '88%', status: 'Optimal' },
  ]
};
}
};

```

5. KCM Competency Radar Component (src/components/CompetencyRadar.jsx)

Visualizes the delta between required KCM baseline proficiency and current real-world workforce scores:

```

import React from 'react';
import { Card, CardContent, Typography, Box } from '@mui/material';
import {
  Radar, RadarChart, PolarGrid, PolarAngleAxis,
  PolarRadiusAxis, ResponsiveContainer, Legend, Tooltip
} from 'recharts';

export default function CompetencyRadar({ data }) {
  return (
    <Card elevation={1} sx={{ height: '100%' }}>
      <CardContent>
        <Typography variant="h6" gutterBottom>
          KCM Competency Gap Analysis (Workforce vs Required)
        </Typography>
        <Typography variant="body2" color="text.secondary" sx={{ mb: 2 }}>
          Evaluation mapped to Capacity Building Commission Functional Standards
        </Typography>
        <Box sx={{ width: '100%', height: 320 }}>
          <ResponsiveContainer>
            <RadarChart data={data}>
              <PolarGrid stroke="#e2e8f0" />
              <PolarAngleAxis dataKey="subject" tick={{ fill: '#475569', fontSize: 11 }} />
              <PolarRadiusAxis angle={30} domain={[0, 100]} />
              <Radar
                name="Required Threshold"
                dataKey="required"

```

```

        stroke="#0c2340"
        fill="#0c2340"
        fillOpacity={0.15}
      />
      <Radar
        name="Current Workforce Level"
        dataKey="current"
        stroke="#e65100"
        fill="#e65100"
        fillOpacity={0.45}
      />
      <Legend wrapperStyle={{ paddingTop: 10 }} />
      <Tooltip />
    </RadarChart>
  </ResponsiveContainer>
</Box>
</CardContent>
</Card>
);
}

```

6. Division Readiness Component (src/components/DivisionReadiness.jsx)

Provides MoSPI institutional leaders with visibility into divisional needs across NSO operational wings:

```

import React from 'react';
import { Card, CardContent, Typography, Box } from '@mui/material';
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid,
  Tooltip, Legend, ResponsiveContainer
} from 'recharts';

export default function DivisionReadiness({ data }) {
  return (
    <Card elevation={1} sx={{ height: '100%' }}>
      <CardContent>
        <Typography variant="h6" gutterBottom>
          MoSPI Divisional Training Demand
        </Typography>
        <Typography variant="body2" color="text.secondary" sx={{ mb: 2 }}>
          Headcount distribution: Fully Proficient vs Upskilling Required
        </Typography>
        <Box sx={{ width: '100%', height: 320 }}>
          <ResponsiveContainer>
            <BarChart data={data}>
              <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" />
              <XAxis dataKey="name" tick={{ fontSize: 12 }} />
              <YAxis />
              <Tooltip />
            </BarChart>
          </ResponsiveContainer>
        </Box>
      </CardContent>
    </Card>
  );
}

```

```

        <Legend />
        <Bar dataKey="proficient" name="Competency Cleared" fill="#0c2340" radius={[4, 4, 0,
0]} />
        <Bar dataKey="needTraining" name="Active Gap Detected" fill="#e65100" radius={[4,
4, 0, 0]} />
      </BarChart>
    </ResponsiveContainer>
  </Box>
</CardContent>
</Card>
);
}

```

7. Master Admin View (src/views/AdminDashboard.jsx)

Combines KPI metrics, radar charts, and state readiness tables into a single scannable view:

```

import React, { useEffect, useState } from 'react';
import {
  Box, Grid, Card, CardContent, Typography, Table,
  TableBody, TableCell, TableContainer, TableHead, TableRow, Chip
} from '@mui/material';
import AssessmentIcon from '@mui/icons-material/Assessment';
import WarningAmberIcon from '@mui/icons-material/WarningAmber';
import VerifiedUserIcon from '@mui/icons-material/VerifiedUser';
import SchoolIcon from '@mui/icons-material/School';
import CompetencyRadar from '../components/CompetencyRadar';
import DivisionReadiness from '../components/DivisionReadiness';
import { fetchAdminMetrics } from '../api/adminClient';

export default function AdminDashboard() {
  const [metrics, setMetrics] = useState(null);

  useEffect(() => {
    fetchAdminMetrics().then(setMetrics);
  }, []);

  if (!metrics) return <Typography sx={{ p: 4 }}>Loading MoSPI Workforce
Metrics...</Typography>;

  return (
    <Box sx={{ p: 4, bgcolor: '#f8f9fa', minHeight: '100vh' }}>
      <Box sx={{ mb: 4 }}>
        <Typography variant="h5" color="primary">
          MoSPI Capacity Building & Competency Intelligence Command
        </Typography>
        <Typography variant="body2" color="text.secondary">
          Integrated Governance Monitor • Karmayogi (iGOT) & NSSTA
Framework[span_5](start_span)[span_5](end_span)
        </Typography>

```

```

</Box>

{/* KPI Cards Grid */}
<Grid container spacing={3} sx={{ mb: 4 }}>
  <Grid item xs={12} sm={6} md={3}>
    <Card elevation={1}>
      <CardContent sx={{ display: 'flex', alignItems: 'center' }}>
        <AssessmentIcon color="primary" sx={{ fontSize: 40, mr: 2 }} />
        <Box>
          <Typography variant="overline" color="text.secondary">Total Assessed</Typography>
          <Typography variant="h5">{metrics.kpi.totalAssessed.toLocaleString()}</Typography>
        </Box>
      </CardContent>
    </Card>
  </Grid>
  <Grid item xs={12} sm={6} md={3}>
    <Card elevation={1}>
      <CardContent sx={{ display: 'flex', alignItems: 'center' }}>
        <VerifiedUserIcon color="primary" sx={{ fontSize: 40, mr: 2 }} />
        <Box>
          <Typography variant="overline" color="text.secondary">Avg.
Readiness</Typography>
          <Typography variant="h5">{metrics.kpi.averageReadiness}%</Typography>
        </Box>
      </CardContent>
    </Card>
  </Grid>
  <Grid item xs={12} sm={6} md={3}>
    <Card elevation={1}>
      <CardContent sx={{ display: 'flex', alignItems: 'center' }}>
        <WarningAmberIcon color="secondary" sx={{ fontSize: 40, mr: 2 }} />
        <Box>
          <Typography variant="overline" color="text.secondary">Critical Gaps</Typography>
          <Typography variant="h5" color="secondary">{metrics.kpi.criticalSkillGaps}
Domains</Typography>
        </Box>
      </CardContent>
    </Card>
  </Grid>
  <Grid item xs={12} sm={6} md={3}>
    <Card elevation={1}>
      <CardContent sx={{ display: 'flex', alignItems: 'center' }}>
        <SchoolIcon color="primary" sx={{ fontSize: 40, mr: 2 }} />
        <Box>
          <Typography variant="overline" color="text.secondary">Scheduled
Workshops</Typography>
          <Typography variant="h5">{metrics.kpi.nsstaWorkshopsScheduled}</Typography>
        </Box>
      </CardContent>
    </Card>
  </Grid>

```

```

        </CardContent>
      </Card>
    </Grid>
  </Grid>

  { /* Charts Grid */ }
  <Grid container spacing={3} sx={{ mb: 4 }}>
    <Grid item xs={12} md={6}>
      <CompetencyRadar data={metrics.kcmRadar} />
    </Grid>
    <Grid item xs={12} md={6}>
      <DivisionReadiness data={metrics.divisionData} />
    </Grid>
  </Grid>

  { /* State / UT Tracking Table */ }
  <Card elevation={1}>
    <CardContent>
      <Typography variant="h6" gutterBottom>State & UT Statistical Bureau
Implementation</Typography>
      <TableContainer>
        <Table size="small">
          <TableHead>
            <TableRow>
              <TableCell><strong>State / UT</strong></TableCell>
              <TableCell><strong>Active Staff</strong></TableCell>
              <TableCell><strong>Completion Rate</strong></TableCell>
              <TableCell><strong>Status</strong></TableCell>
            </TableRow>
          </TableHead>
          <TableBody>
            {metrics.stateRankings.map((row) => (
              <TableRow key={row.state} hover>
                <TableCell>{row.state}</TableCell>
                <TableCell>{row.totalStaff}</TableCell>
                <TableCell>{row.completionRate}</TableCell>
                <TableCell>
                  <Chip
                    size="small"
                    label={row.status}
                    color={row.status === 'Optimal' ? 'primary' : row.status === 'Review Needed' ?
'warning' : 'error'}
                  />
                </TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </TableContainer>
    </CardContent>
  </Card>

```

```
        </TableContainer>
      </CardContent>
    </Card>
  </Box>
);
}
```

8. Verification Checklist for Member 5

- * Recharts Rendering: Verify that both the Radar and Stacked Bar charts adjust properly across full-screen presentation monitors without clipping data labels.
- * Graceful Degradation: Disconnect the backend server and ensure the dashboard falls back to internal mock data cleanly without throwing an unhandled runtime error.
- * Data Parity with Member 1: Coordinate with Member 1 (Priyash) to ensure the API response keys (required, current, proficient, needTraining) match the live PostgreSQL aggregation query.

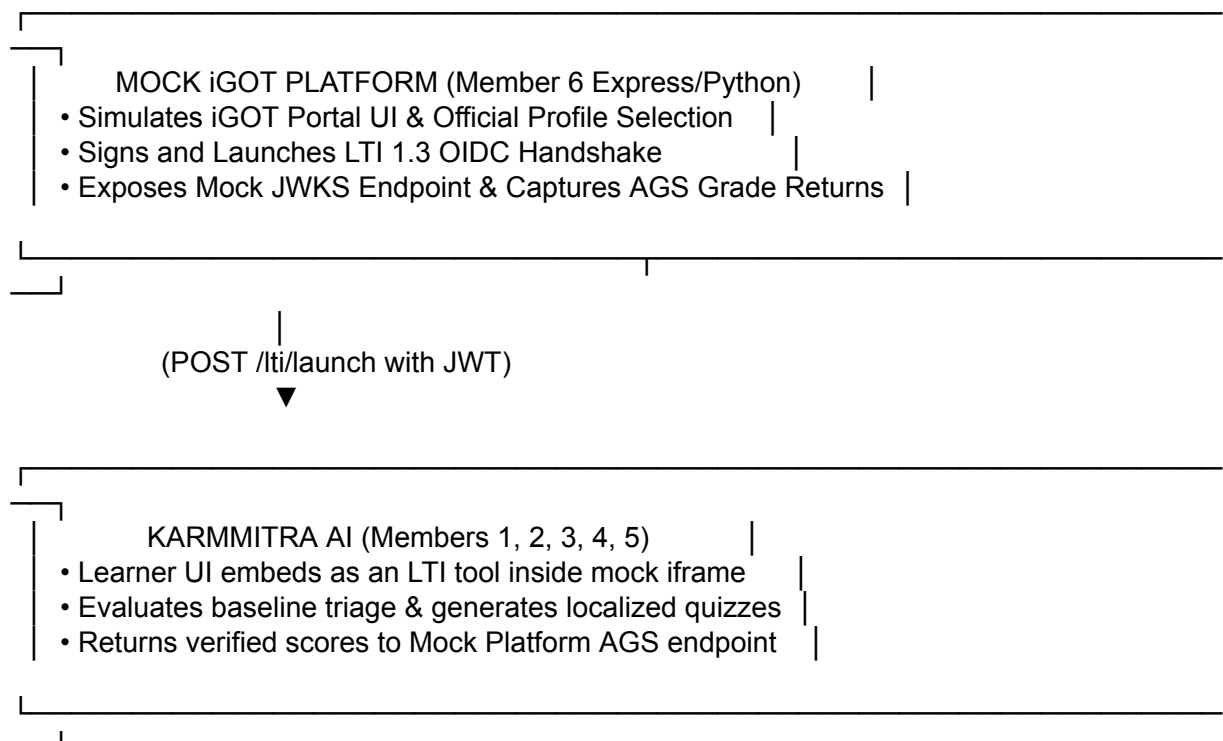
Step-by-Step Execution Guide: Member 6

Role: Mock Ecosystem, Synthetic Data Seeding & Integration QA

Technical Level: Easy / Foundational (Node.js / Express or Python, JSON data modeling, QA testing)

Deliverable: A lightweight local mock server that simulates the iGOT Karmayogi LMS platform, acts as an LTI 1.3 launch initiator, creates synthetic MoSPI official profiles with FRAC roles, captures AGS grade passback requests, and provides end-to-end integration tests for the team.

LOCAL SIMULATION ECOSYSTEM



1. System Architecture & Directory Layout

mock-igot-platform/

```
├── package.json
├── server.js           # Express server simulating iGOT Karmayogi
├── certs/
│   ├── platform_private.key # RSA key for signing test LTI launch tokens
│   └── platform_public.key  # Exposed via /.well-known/jwks.json
├── data/
│   ├── synthetic_users.json # Seeded MoSPI statistical officer personas
│   └── course_catalog.json  # Mocked 70:20:10 iGOT/NSSTA course metadata
├── public/
│   └── index.html          # Portal UI to select a user and launch KarmMitra
└── test_runner.py          # Automated health and integration testing script
```

2. Dependencies & Scaffolding (package.json)

Use a lightweight Express setup to simulate the government platform without heavy infrastructure:

```
mkdir mock-igot-platform
```

```
cd mock-igot-platform
```

```
npm init -y
```

```
npm install express cors jsonwebtoken node-rsa body-parser
```

Ensure your package.json contains:

```
{
  "name": "mock-igot-platform",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "body-parser": "^1.20.2",
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "node-rsa": "^1.1.1"
  }
}
```

3. Platform RSA Key Generation

Generate a dedicated keypair for the mock platform inside certs/:

```
mkdir -p certs
```

```
openssl genrsa -out certs/platform_private.key 2048
```

```
openssl rsa -in certs/platform_private.key -pubout -out certs/platform_public.key
```

4. Synthetic Data Seeding (data/synthetic_users.json)

To present realistic MoSPI scenarios to the judges, seed diverse user profiles representing real

operational divisions (FOD, SDRD, NAD):

```
[
  {
    "sub": "MOSPI_OFFICER_001",
    "name": "Rajesh Kumar Sharma",
    "email": "rajesh.sharma@mospi.gov.in",
    "division": "Field Operations Division (FOD)",
    "frac_role": "MOSPI_FOD_INV_01",
    "designation": "Junior Statistical Officer (JSO)",
    "state": "Uttar Pradesh",
    "history_count": 0,
    "scenario": "Brand New Recruit (Cold-Start Triage Test)"
  },
  {
    "sub": "MOSPI_OFFICER_002",
    "name": "Priyanka Sen",
    "email": "priyanka.sen@mospi.gov.in",
    "division": "Survey Design & Research Division (SDRD)",
    "frac_role": "MOSPI_SDRD_ANL_02",
    "designation": "Senior Statistical Officer (SSO)",
    "state": "West Bengal",
    "history_count": 8,
    "scenario": "Experienced Data Modeler (Advanced Gap Analysis)"
  },
  {
    "sub": "MOSPI_OFFICER_003",
    "name": "M. Ananthakrishnan",
    "email": "m.ananth@mospi.gov.in",
    "division": "National Accounts Division (NAD)",
    "frac_role": "MOSPI_NAD_ECO_01",
    "designation": "Director (National Accounts)",
    "state": "Tamil Nadu",
    "history_count": 14,
    "scenario": "Macro-economic Specialist (CPI/IIP Indices)"
  }
]
```

5. Mock iGOT Server Implementation (server.js)

This script handles the platform-side authorization flow, serves the public keys, signs JWTs, and receives score passbacks from KarmMitra:

```
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const jwt = require('jsonwebtoken');
const fs = require('fs');
const path = require('path');

const app = express();
```

```

const PORT = 9000;

app.use(cors());
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static('public'));

const privateKey = fs.readFileSync(path.join(__dirname, 'certs/platform_private.key'), 'utf8');
const publicKey = fs.readFileSync(path.join(__dirname, 'certs/platform_public.key'), 'utf8');
const users = JSON.parse(fs.readFileSync(path.join(__dirname, 'data/synthetic_users.json'), 'utf8'));

// 1. JWKS Endpoint (Used by Member 3 to verify launch signatures)
app.get('/.well-known/jwks.json', (req, res) => {
  res.json({
    keys: [
      {
        kty: 'RSA',
        alg: 'RS256',
        use: 'sig',
        kid: 'mock-platform-key-1',
        key: publicKey
      }
    ]
  });
});

// 2. Fetch seeded official personas for the demo portal UI
app.get('/api/users', (req, res) => {
  res.json(users);
});

// 3. OIDC Launch Simulator: Generates signed LTI 1.3 JWT
app.post('/platform/launch', (req, res) => {
  const { userId } = req.body;
  const user = users.find(u => u.sub === userId) || users[0];

  const payload = {
    iss: 'https://igotkarmayogi.gov.in',
    sub: user.sub,
    aud: 'karmmitra_sih26101_tool',
    exp: Math.floor(Date.now() / 1000) + 3600,
    iat: Math.floor(Date.now() / 1000),
    name: user.name,
    email: user.email,
    'https://purl.imsglobal.org/spec/lti/claim/message_type': 'LtiResourceLinkRequest',
    'https://purl.imsglobal.org/spec/lti/claim/version': '1.3.0',
    'https://purl.imsglobal.org/spec/lti/claim/custom': {

```

```

    frac_role: user.frac_role,
    division: user.division,
    state: user.state
  },
  'https://purl.imsglobal.org/spec/lti-agis/claim/endpoint': {
    scope: ['https://purl.imsglobal.org/spec/lti-agis/scope/score'],
    lineitem: `http://localhost:${PORT}/platform/ags/lineitem/competency_assessment`
  }
};

const idToken = jwt.sign(payload, privateKey, { algorithm: 'RS256', keyid: 'mock-platform-key-1'
});

// Auto-submitting HTML form simulating an LMS POST redirect to KarmMitra backend
res.send(`
<html>
  <body onload="document.forms[0].submit()">
    <form method="POST" action="http://localhost:8000/lti/launch">
      <input type="hidden" name="id_token" value="${idToken}" />
    </form>
    <p>Connecting to KarmMitra AI Sovereign Engine via LTI 1.3 Handshake...</p>
  </body>
</html>
`);
});

// 4. LTI 1.3 Assignment and Grade Services (AGS) Receiver
app.post('/platform/ags/lineitem/competency_assessment/scores', (req, res) => {
  const gradePayload = req.body;
  console.log("\n[MOCK iGOT PLATFORM] Successfully received AGS Grade Passback:");
  console.log(JSON.stringify(gradePayload, null, 2));

  res.status(200).json({
    status: 'success',
    message: 'Grade recorded securely into simulated iGOT gradebook',
    timestamp: new Date().toISOString()
  });
});

app.listen(PORT, () => {
  console.log(`Mock iGOT Karmayogi Platform listening on http://localhost:${PORT}`);
});

```

6. Portal Simulator Interface (public/index.html)

A testing screen that lets the team pick an official persona and launch KarmMitra inside a simulated iGOT frame during live hackathon demos:

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
  <meta charset="UTF-8">
  <title>iGOT Karmayogi • Simulation Gateway</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, sans-serif; background: #eef2f6; margin: 0; padding:
20px; }
    .card { background: white; max-width: 650px; margin: 40px auto; padding: 30px;
border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
    h2 { color: #0c2340; margin-top: 0; }
    select, button { width: 100%; padding: 12px; margin-top: 15px; font-size: 15px; border-radius:
6px; }
    button { background: #0c2340; color: white; border: none; font-weight: bold; cursor: pointer; }
    button:hover { background: #e65100; }
    .badge { display: inline-block; background: #e2e8f0; padding: 4px 8px; border-radius: 4px;
font-size: 12px; margin-top: 10px; }
  </style>
</head>
<body>
  <div class="card">
    <h2>iGOT Karmayogi Portal Simulator</h2>
    <p>Select a MoSPI statistical official persona to test the LTI 1.3 launch handshake and
baseline triage flow:</p>

    <form method="POST" action="/platform/launch">
      <label for="userId"><strong>Select Active Civil Servant:</strong></label>
      <select name="userId" id="userId">
        <option value="MOSPI_OFFICER_001">Rajesh Sharma (JSO - Field Operations Division)
[Cold Start]</option>
        <option value="MOSPI_OFFICER_002">Priyanka Sen (SSO - Survey Design
Division)</option>
        <option value="MOSPI_OFFICER_003">M. Ananthakrishnan (Director - National
Accounts)</option>
      </select>
      <div class="badge">Simulating: LTI 1.3 Core • IMS Global Security • Sunbird Backend</div>
      <button type="submit">Launch KarmMitra Embedded AI Plugin</button>
    </form>
  </div>
</body>
</html>

```

7. Automated Integration Test Script (test_runner.py)

Member 6 uses this script to verify that the whole team's microservices are healthy before presentations or judge evaluation rounds:

```

import requests
import sys

```

```

SERVICES = {
  "Member 1: FastAPI Gateway": "http://localhost:8000/health",

```

```

"Member 2: Sovereign AI Engine": "http://localhost:11434/api/tags",
"Member 3: LTI 1.3 JWKS Endpoint": "http://localhost:8000/lti/jwks.json",
"Member 4: Learner UI": "http://localhost:3000",
"Member 5: Admin UI": "http://localhost:5173",
"Member 6: Mock iGOT Platform": "http://localhost:9000"
}

def run_preflight_checks():
    print("=" * 60)
    print("KARMMITRA AI • SYSTEM HEALTH & PREFLIGHT CHECK")
    print("=" * 60)

    all_healthy = True
    for name, endpoint in SERVICES.items():
        try:
            res = requests.get(endpoint, timeout=3.0)
            if res.status_code in [200, 302, 303]:
                print(f"[PASSED] {name} is active -> {endpoint}")
            else:
                print(f"[FAILED] {name} returned status code: {res.status_code}")
                all_healthy = False
        except requests.exceptions.RequestException:
            print(f"[OFFLINE] {name} is unreachable at {endpoint}")
            all_healthy = False

    print("=" * 60)
    if all_healthy:
        print("ALL 6 SUBSYSTEMS OPERATIONAL. READY FOR DEMO.")
    else:
        print("WARNING: Some services are offline. Resolve before presenting.")
    print("=" * 60)

if __name__ == "__main__":
    run_preflight_checks()

```

8. Verification Checklist for Member 6

- * Standalone Execution: Run npm start inside mock-igot-platform and verify that visiting <http://localhost:9000> renders the portal simulator cleanly.
- * Handshake Verification: Click "Launch KarmMitra Embedded AI Plugin" and confirm it successfully POSTs the signed token to Member 3's /lti/launch endpoint.
- * Passback Inspection: Ensure the terminal outputs the captured [MOCK iGOT PLATFORM] Successfully received AGS Grade Passback log when a user completes the quiz in Member 4's UI.
- * Offline Demo Readiness: Record a 2-minute clean screen walkthrough of this end-to-end launch and scoring flow to serve as a backup if Wi-Fi drops during the SIH evaluation.